Lab 3 Report

Compile and Run Commands:

Run from command line using the python3 command to explicitly specify that the script should be executed using the Python 3.x interpreter.

Command line: python3 predictor.py

Prediction Rate Comparison:

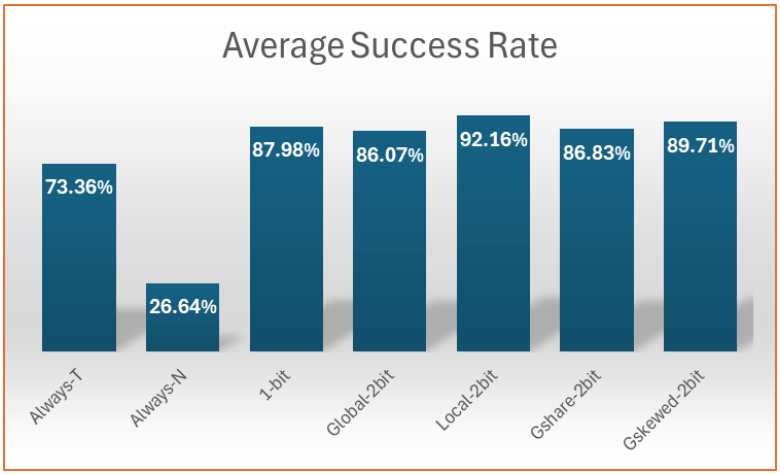
Results from predictor.py

```
arch16@chicago:~/lab3$ python3 predictor.py

Processing: branch-vortex.tr...
Predictor Type       | Accuracy
-----------------------------------
Always-T             |    67.07%
Always-N             |    32.93%
1-bit                |    87.43%
Global-2bit          |    83.14%
Local-2bit           |    91.19%
Gshare-2bit          |    85.04%
Gskewed-2bit         |    88.60%

Processing: branch-bzip2.tr...
Predictor Type       | Accuracy
-----------------------------------
Always-T             |    79.05%
Always-N             |    20.95%
1-bit                |    91.48%
Global-2bit          |    89.16%
Local-2bit           |    94.23%
Gshare-2bit          |    89.29%
Gskewed-2bit         |    91.83%

Processing: branch-art.tr...
Predictor Type       | Accuracy
-----------------------------------
Always-T             |    73.97%
Always-N             |    26.03%
1-bit                |    85.03%
Global-2bit          |    85.90%
Local-2bit           |    91.07%
Gshare-2bit          |    86.16%
Gskewed-2bit         |    88.70%
arch16@chicago:~/lab3$
```

Results Comparison Table

	Success Rate by File			
Type	Vortx.tr	bzip2.tr	art.tr	Average
Always-T	67.07%	79.05%	73.97%	73.36%
Always-N	32.93%	20.95%	26.03%	26.64%
1-bit	87.43%	91.48%	85.03%	87.98%
Global-2bit	83.14%	89.16%	85.90%	86.07%
Local-2bit	91.19%	94.23%	91.07%	92.16%
Gshare-2bit	85.04%	89.29%	86.16%	86.83%
Gskewed-2bit	88.60%	91.83%	88.70%	89.71%

Graph of Average Success Rate by Type



Analysis of Code:

predictor.py comfortably operates within the storage limitations budgeted by the lab. In fact, the current configuration uses less than 10% of the allocated budget.

Breakdown of storage requirements for most complex predictor (Gskewed) and local history components:

History Registers (The "1K" Budget)

predictor.py uses a 10-bit Global History Register (GHR) and a 1024-entry Local History Table (LHT).
GHR: 10 bits.
LHT: 1024 entries × 10 bits = 10,240 bits (1.25 KB).
Total: ~1.26 KB. Slightly over the "1K" for Local history if strictly enforced, but well within the overall 65K budget.

Predictor Tables (The "64K" Budget)

The storage is calculated by Number of Entries × Bits per Entry:

Predictor	Calculation	Bits	Bytes
1-bit PHT	1024 entries × 1 bit	1,024	128 B
Global 2-bit	1024 entries × 2 bits	2,048	256 B
Local 2-bit	1024 entries × 2 bits	2,048	256 B
Gshare	1024 entries × 2 bits	2,048	256 B
Gskewed	3 banks × 1024 entries × 2 bits	6,144	768 B

Summary:

The largest predictor (Gskewed) uses only 768 Bytes of the available 65,536 Bytes (64K).

Recommendation to improve accuracy:

Increase the PHT_SIZE and HIST_BITS. By increasing PHT_SIZE to 16,384 (14-bit index) and HIST_BITS to 14, a Gshare predictor would use: 16,384 x 2 bits = 32,769 bits = 4KB.

It would be possible to increase to a 256K-entry table before hitting the 64KB limit. Utilizing more of the budget would significantly reduce aliasing and improve accuracy scores.

Analysis of Results:

1. *Base Line Results: Always Taken (73.36%) vs. Always Not Taken (26.64%)*

   The baseline of the static predictors came in as expected with Always Taken significantly outperforming Always Not Taken since most branches in typical code are loops that go back and are Taken. Based on these results we would want our predictors to outperform 73.36% as a measure of usefulness.

2. *Local-2bit Results: (92.16%)*

   In this exercise, using the LHT (Local History Table) to remember what a specific branch did the last time, yielded the best average results (92.16%). Even with a small 1024-entry table, it identified these individual patterns much better than a global register. The global register may have a tendency to get "polluted" by every other branch in the program.

3. *Global vs. Gshare vs. Gskewed*

   Global-2bit (86.07%): This performs worse than the 1-bit bimodal (87.98%) in some cases because the 10-bit GHR (Global History Register) changes every time any branch is taken. This can lead to aliasing, where two completely different branches overwrite each other's data in the PHT.

   Gshare-2bit (86.83%): We see a slight bump here because in our predictor, pc_idx ^ ghr helps randomize the index. This helps reduce the chance of two branches colliding in the same PHT entry.

Gskewed-2bit (89.71%): This was second most successful predictor. By using three different hash functions and a majority vote, our predictor effectively ignores a collision if it only happens in one of the three banks.

4. *Why does 1-bit (87.98%) beat Global (86.07%)?*

   This is likely due to the small tables. A 1-bit bimodal predictor is very stable; it only changes when a branch actually flips. A Global predictor with only 10 bits of history has a history pattern that changes so frequently that the 2-bit counters rarely have time to attain consistency for a specific branch before they are overwritten by another.

5. *Key Points*

   The Local-2bit predictor is winning because the trace files (vortex, bzip2, art) contain many loops and predictable local patterns. To make the Gshare or Gskewed predictors beat the Local one, we would need to use more of that 64K budget to increase the HIST_BITS and PHT_SIZE, allowing them to track much longer, more complex global patterns